# 02 06 Tag 2 ELK-Stack und Suricata

**Rolle: IT-Security-Analyst, AlphaTech GmbH**

## Thema: Netzwerk-Traffic überwachen, analysieren, normal vs. verdächtig unterscheiden

## Datum: 2026-03-24

## 1) Kibana Web-Interface erkunden

### Ziel

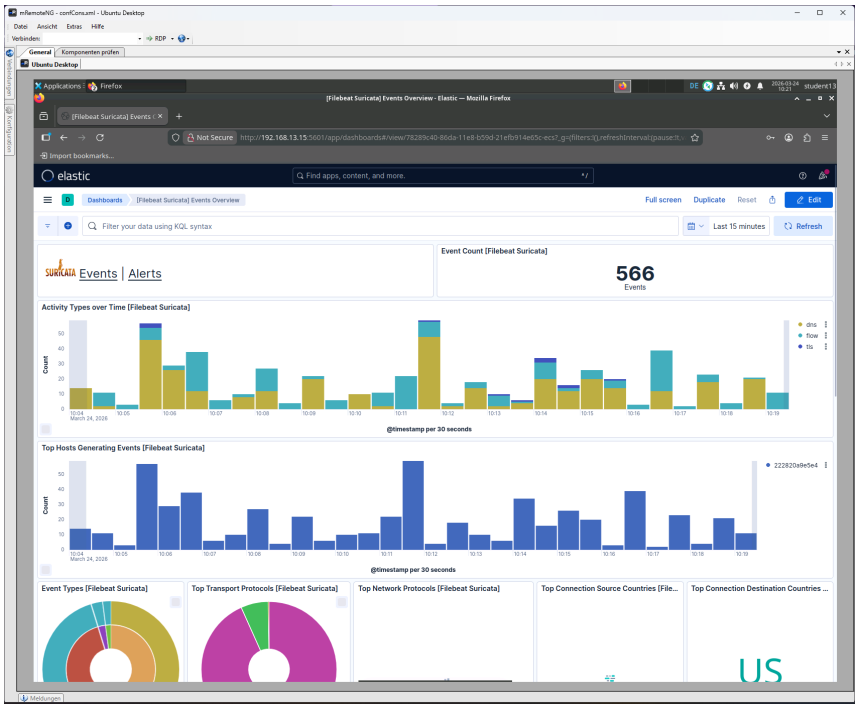
Suricata-Daten im Dashboard finden und Grundaufbau verstehen.

### Schritte

1. Kibana öffnen: http://192.168.13.15:5601
2. Navigieren zu: Analytics → Dashboard
3. Suche: Suricata
4. Öffnen: Suricata Overview Dashboard

### Was du prüfen sollst

- Event-Anzahl über Zeit
- Top Source IPs / Destination IPs
- Top Ports / Protokolle

- Alert-Übersicht 

- [SCREENSHOT_02: Widgets mit Top IPs + Top Ports]

---

## 2) Begriffe klären (Event, Alert, Incident)

### Event

Ein Event ist ein einzelner Log-Eintrag zu Netzwerkaktivität (z. B. DNS-Query, TCP-Flow, HTTP-Request, Metadaten, Statistik-Event).

**Beispiele aus den Daten:**

- event_type: dns (DNS query/answer)
- event_type: flow (Verbindungsfluss)
- event_type: stats (Suricata Performance-/Zählerdaten)

### Alert

Ein Alert ist ein Event, das eine Suricata-Regel getriggert hat (Signatur/Erkennung).

- Alerts entstehen nur, wenn Regelbedingungen passen.
- Beispiel: Exploit-Signatur, Portscan-Muster, bekannte IOC-Domain, verdächtiger Payload.

## Incident

Ein Incident ist ein bestätigter Sicherheitsvorfall nach Analyse/Korrelation. Nicht jeder Alert = Incident.

---

## Alert vs. Incident – Unterschied

- Alert: technische Warnung (kann False Positive sein)
- Incident: verifizierter Vorfall mit Auswirkung/Risiko + Reaktion

---

## Erzeugt jedes Event automatisch einen Alert?

Nein. Technisch gilt:

1. Suricata erzeugt viele rohe Events (flow/dns/stats/etc.).
2. Alerts entstehen nur bei Regel-Match.
3. Ohne Match bleibt es ein normales Event.

---

## 3) Netzwerk-Baseline verstehen (10–15 Min Beobachtung)

## Auswertung aus meinem 15-Minuten-Log

## Normalwerte (Baseline)

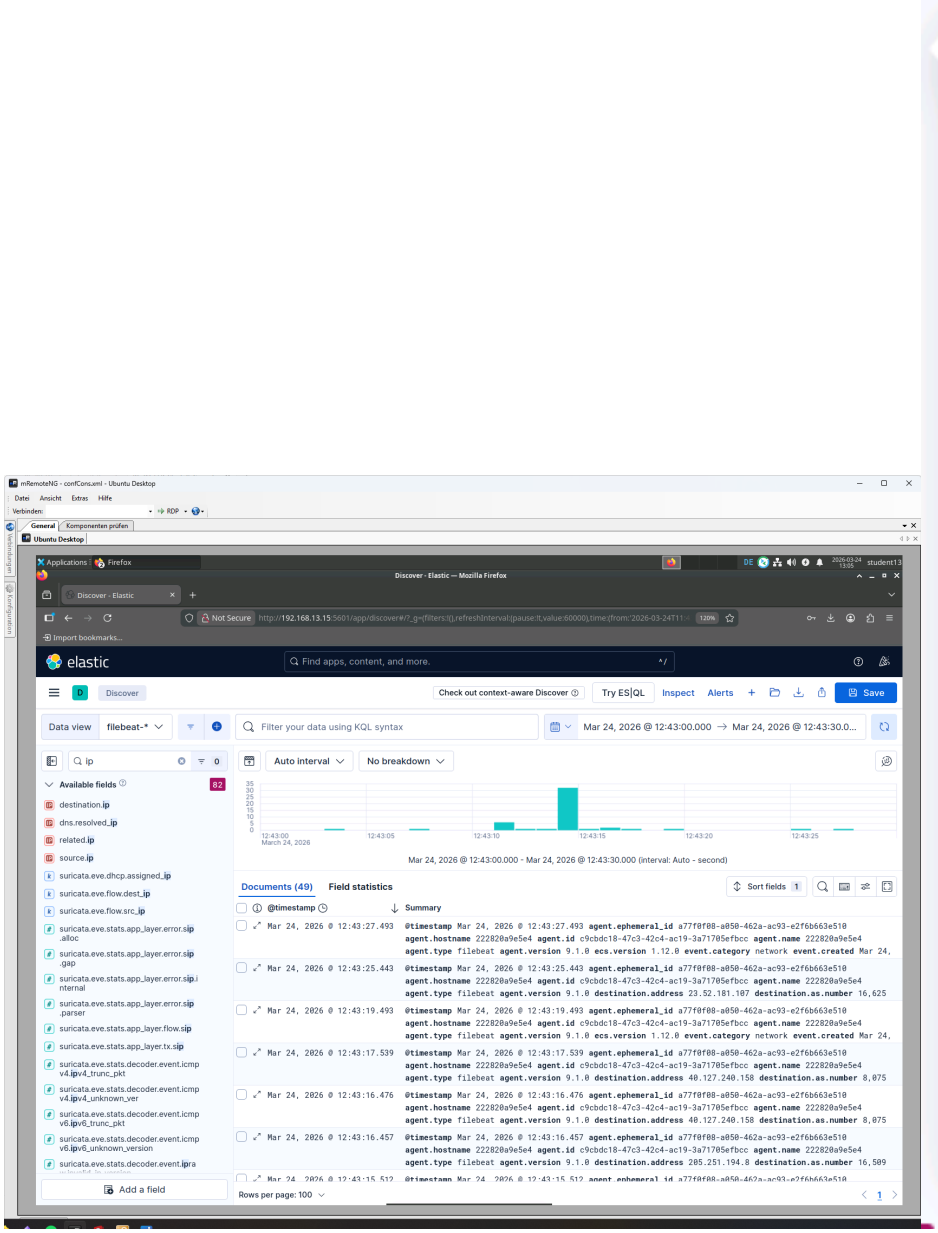
- Events gesamt: ~20.378
- Ø Events/Minute: ~1.358
- Packet Drops: 0 (sehr gut)
- Dominantes Protokoll: DNS (Port 53, UDP stark dominant)
- Wichtige interne Hosts: 192.168.13.20, 192.168.13.10, 192.168.13.2
- Häufige externe Ziele: Fastly, Google DNS-nahe Infrastruktur, VeriSign, SWITCH

## Häufige Ports

- 53/udp (DNS) – sehr hoch
- 5601/tcp (Kibana) – intern auffällig vorhanden
- 443/tcp (HTTPS) – normal vorhanden
- 138/udp (NetBIOS Broadcast) – vorhanden, beobachtungswürdig

## Baseline-Satz

„Normaler Traffic in dieser Umgebung zeigt hohes DNS-Aufkommen (~1.3k Events/min), primär interne Hosts mit externen Resolver-/CDN-Zielen, ohne Packet Loss und ohne bestätigte Sicherheitsvorfälle.“


destination.p
Port of the dest
Top value
53
443
80
49234
Calculated from
elastic
Discover
Data view
filebeat-*
Available fields
destination.ip
dns.resolved_ip
related.ip
source.ip
Documents (49)
Field statistics
Add a field

---

## 4) Log-Analyse in Discover

**Navigation**

- Analytics → Discover
- Dataset/Filter: event.dataset: suricata.eve

## A) DNS-Events analysieren

**Filter-Idee:**

- suricata.eve.event_type: dns

**Was sichtbar war:**

- Viele query + answer Events
- Häufige Domains z. B. ntp.ubuntu.com, com
- Antwortcodes oft NOERROR
- A/AAAA Antworten mit mehreren IPs

**Kurzbewertung:**

In Summe wirkt DNS-Last hoch, aber größtenteils plausibel (Resolver/CDN/Time-Sync).

---

## B) HTTP/HTTPS-Traffic analysieren

**Filter-Ideen:**

- network.transport: tcp AND destination.port: 443
- ggf. suricata.eve.tls.sni:*

**Was sichtbar war:**

- TLS-Flows vorhanden
- Kein offensichtlicher Alert-Bezug im gezeigten Ausschnitt

**Kurzbewertung:**

Normale verschlüsselte Web-Kommunikation.

---

## C) SSH-Verbindungen analysieren

**Filter-Ideen:**

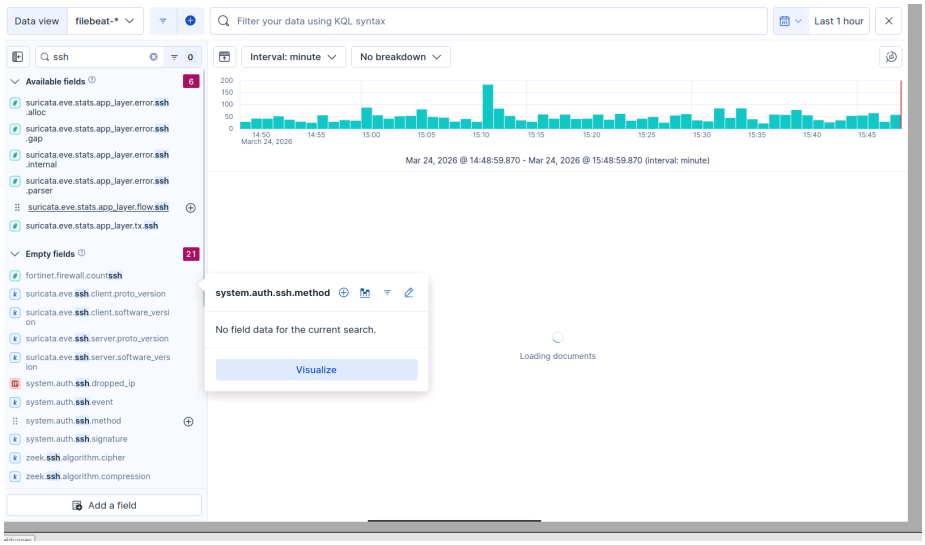
- destination.port: 22 OR source.port: 22
- suricata.eve.event_type: flow AND network.transport: tcp

**Was sichtbar war:**

- In diesem Ausschnitt keine dominante SSH-Aktivität.

**Kurzbewertung:**

Kein akuter SSH-Bruteforce-Hinweis im bereitgestellten Sample.




***

## 5) Top 5 Suricata Alert Signatures analysieren

# Suricata Alert Analyse – 5 Signaturen

---

## 1 ET INFO GNU/Linux APT User-Agent Outbound

### Technische Bedeutung

Erkennt ausgehenden HTTP-Traffic mit einem APT-User-Agent (Debian APT-HTTP/1.3) – typisch für Linux-Paketverwaltung (apt-get, apt upgrade)

### Entstehung

- source: Host 222820a9e5e4 (Kibana-Host 192.168.13.15)
- destination.address: de.archive.ubuntu.com
- url: /ubuntu/pool/main/p/pyopenssl/python3-openssl_23.2.0-1ubuntu0.1_all.deb
- user_agent.original: Debian APT-HTTP/1.3 (2.8.3) non-interactive
- @timestamp: Mar 24, 2026 @ 15:12:03
- 44 Events, primär aus DE

### Einordnung

> [!success] **False Positive / Normal**
> Regulärer Paketdownload

### Ursache

Auf dem Ubuntu-Host wurden Pakete via apt installiert oder aktualisiert. Suricata erkennt den charakteristischen User-Agent und erzeugt einen Info-Alert.

---

## 2 SURICATA AF-PACKET truncated packet

### Technische Bedeutung

Suricata konnte ein Paket über das AF-PACKET-Interface nicht vollständig verarbeiten.

→ Paket wurde **abgeschnitten (truncated)**

### Entstehung

- host.name: 222820a9e5e4
- suricata.eve.in_iface: ens19
- suricata.eve.pkt_src: wire/pcap
- event.category: ["network", "intrusion_detection"]
- event.kind: alert
- event.severity: 3
- Keine source/dest IPs im Log
- 23 Events, ~09:25 Uhr

### Einordnung

> [!warning] **False Positive / Systembedingt**

### Ursache

- Snaplen zu klein

- AF_PACKET Buffer Limit
- Fragmentierung durch Defrag Engine

Suricata kann fragmentierte Pakete größer als MTU reassemblen,
aber wenn das Capture-Limit kleiner ist, werden sie als truncated erkannt :contentReferenceoaicite:0

---

## 3 SURICATA IPv4 truncated packet

### Technische Bedeutung

Ein IPv4-Paket hat eine inkonsistente oder ungültige Länge.

→ Suricata kann es nicht vollständig dekodieren

### Entstehung

- signature_id: 2200003
- interface: ens19
- pkt_src: wire/pcap
- event.severity: 3
- host.name: 222820a9e5e4
- 23 Events, ~09:25 Uhr
- Keine IP-Adressen sichtbar

### Einordnung

> [!warning] **False Positive / Systembedingt**

### Ursache

- Fragmentierte Pakete
- MTU-Mismatch
- Virtualisierung (VM Netzwerkstack)

truncated packets entstehen oft durch MTU-basierte Limits im Capture-System :contentReferenceoaicite:1

---

## 4 SURICATA STREAM excessive retransmissions

### Technische Bedeutung

Ungewöhnlich viele TCP-Retransmissions erkannt

→ Hinweis auf: - Paketverlust - Netzwerkprobleme - selten: Evasion-Techniken

### Entstehung

- host.name: 222820a9e5e4
- interface: ens19
- event.category: ["network", "intrusion_detection"]
- 9 Events

### Einordnung

[!warning] **Wahrscheinlich False Positive**

### Ursache

- Paketverlust durch truncated packets
- TCP resend Verhalten
- mögliche Netzprobleme (MTU, Buffer)

Packet Loss ist ein typischer Auslöser für Retransmissions :contentReferenceoaicite:2

---

## 5 ET INFO Microsoft Connection Test

### Technische Bedeutung

Windows Connectivity Check (NCSI)

→ HTTP Request zu Microsoft-Testservern

### Entstehung

- destination.address: 2.18.64.212 / 2.18.64.208
- @timestamp: Mar 24, 2026 (08:48–09:23)
- agent.hostname: 22282de9e5e4
- alert.category: Misc activity
- 4 Events
- Source Country: DE

### Einordnung

[!success] **False Positive / Normal**

### Ursache

Windows-System prüft Internetverbindung automatisch
(NCSI – Network Connectivity Status Indicator)

---

## Gesamtbewertung

[!summary] Alle 5 Alerts sind **keine Angriffe**, sondern:

- normale Systemaktivität (APT, Microsoft)
- technische Artefakte (Packet Truncation, Retransmissions)

---

## Technisches Problem (wichtig)

Die Alerts 2–4 zeigen ein gemeinsames Problem:

[!warning] **Capture / Netzwerk Problem im Suricata Sensor**

Typische Ursache:

- Snaplen zu klein (Standard ~1500 Byte)
- Defragmentierung erzeugt größere Pakete
- AF_PACKET Buffer zu klein

Suricata zählt solche Fälle als `trunc_pkt` in den Stats :contentReferenceoaicite:3

---

## Empfehlungen

### Fixes

- snaplen auf 65535 erhöhen
- AF_PACKET ring-size erhöhen
- MTU prüfen
- CPU / Performance checken

### Monitoring

- stats.capture.kernel_drops
- stats.decoder.event.afpacket.trunc_pkt

---

## Fazit

| Kategorie | Bewertung |
|---|---|
| Security Risiko | Kein Angriff |
| Systemzustand | Optimierungsbedarf |

| Kategorie | Bewertung |
|---|---|
| Detection Qualität | eingeschränkt |

Hauptproblem ist **nicht Security, sondern Visibility**

---

## Konkrete Beobachtungen aus meinen Daten (für Bewertung)

1. Hoher DNS-Durchsatz (dns_udp sehr hoch) → eher normal für aktive Clients, aber DNS-Tunneling-Check empfohlen.
2. Mehrere TCP-Flows zu Port 5601 (Kibana) mit syn_sent/new/timeout → kann legitimer Zugriff sein, aber auffällig bei Häufung.
3. NetBIOS Broadcast auf UDP/138 → oft Legacy/Windows-Networking; sicherheitstechnisch überwachen.
4. Keine Packet Drops (kernel_drops: 0) → Sensorqualität gut, Datenlage zuverlässig.
5. Alerts vorhanden (detect.alert: 52), viele suppressed (374) → Tuning/Noise-Management relevant.